

Software Construction
Observation
B. Tech AIML

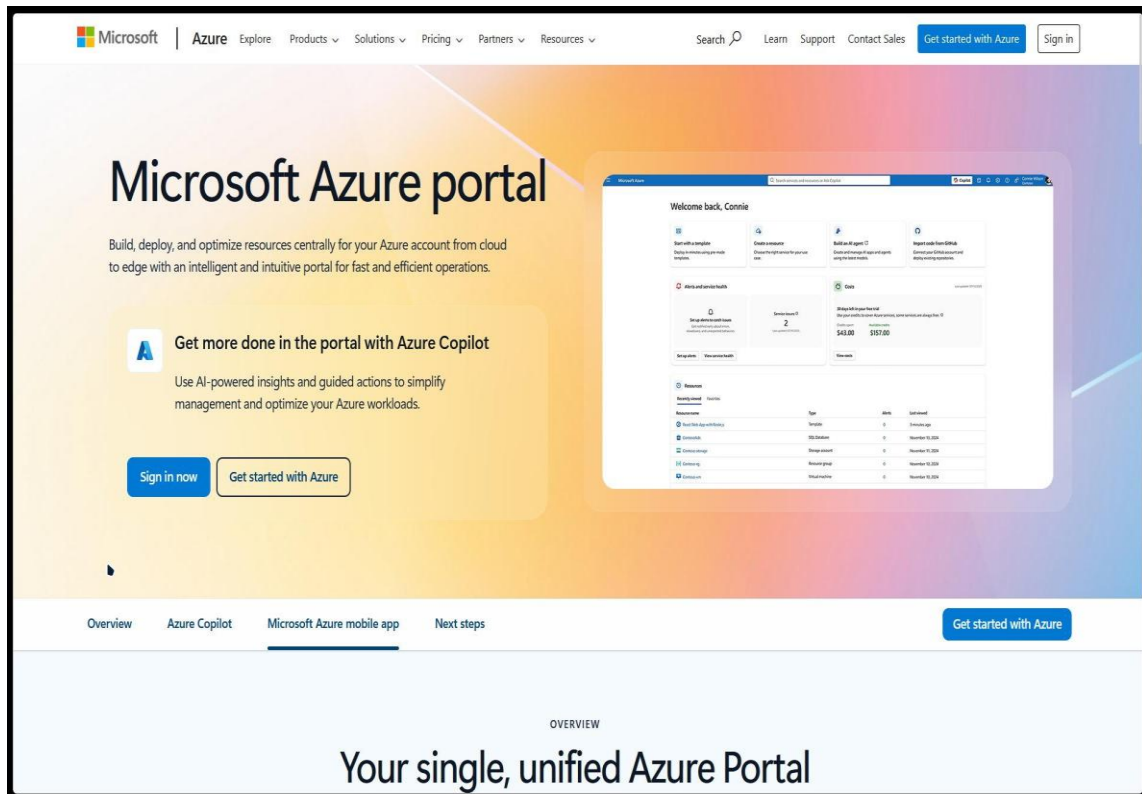
Done By
Harish M , 241501066
Mythrayee V, 241501125

INDEX

S.No.	Date	Title	Page No.	Teacher's Signature Remarks
1.		Azure DevOps Environment Setup		
2.		Azure DevOps Project Setup and User Story Management		
3.		Setting Up Epics, Features, and User Stories for Project Planning		
4.		Sprint Planning		
5.		Poker Estimation		
6.		Designing Class and Sequence Diagrams for Project Architecture		
7.		Designing Architectural and ER Diagrams for Project Structure		
8.		Testing - Test Plans and Test Cases		
9.		Load Testing and Performance Testing		
10.		Github: Project Structure & Naming Conventions		

AZURE DEVOPS ENVIRONMENT SETUP:

1. Open your web browser and go to the Azure website:
<https://azure.microsoft.com/en-us/get-started/azure-portal>.

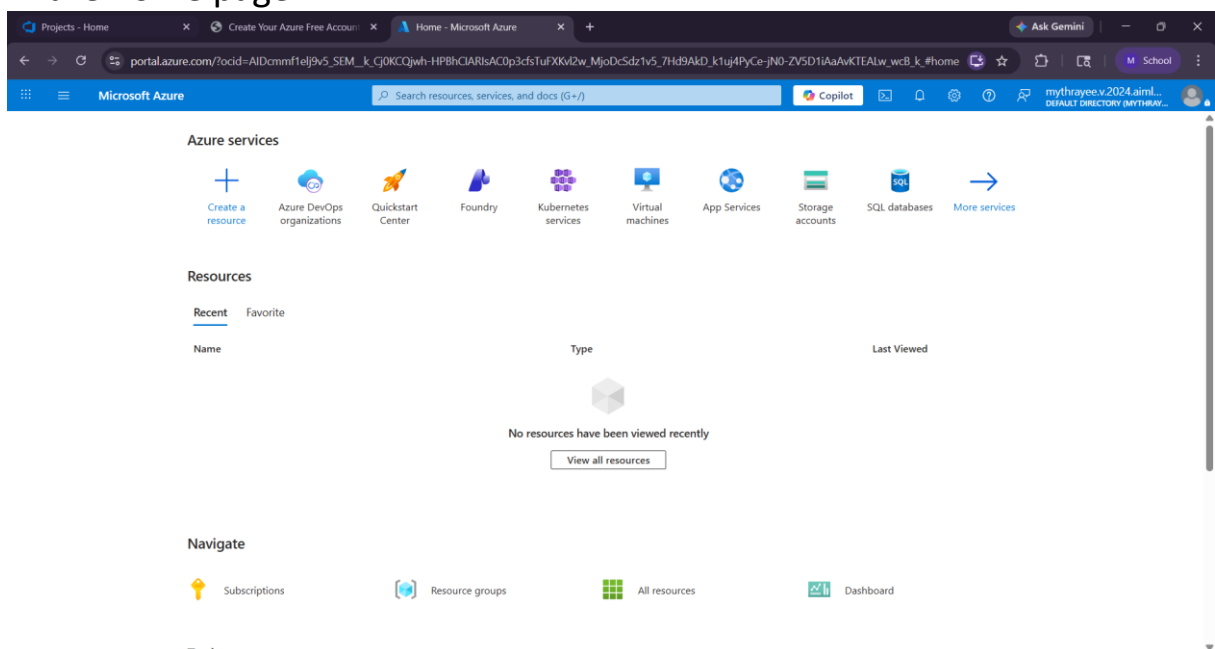


Sign in using your Microsoft account credentials.

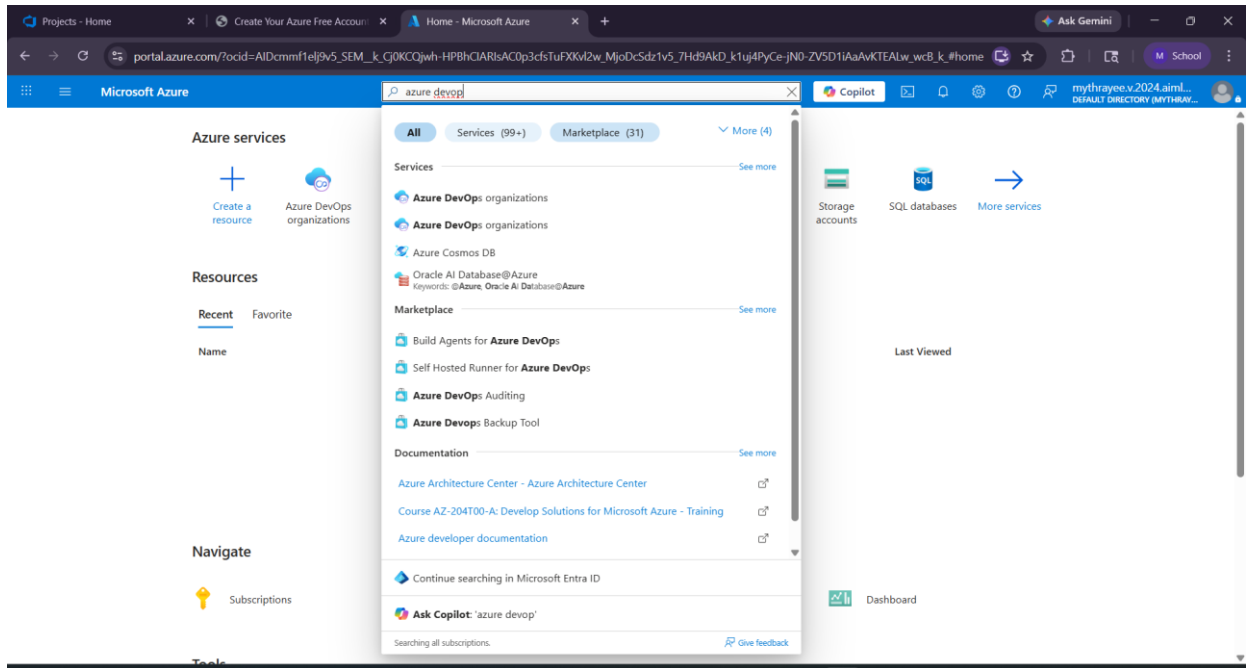
If you don't have a Microsoft account, you can create one here:

<https://signup.live.com/?lic=1>

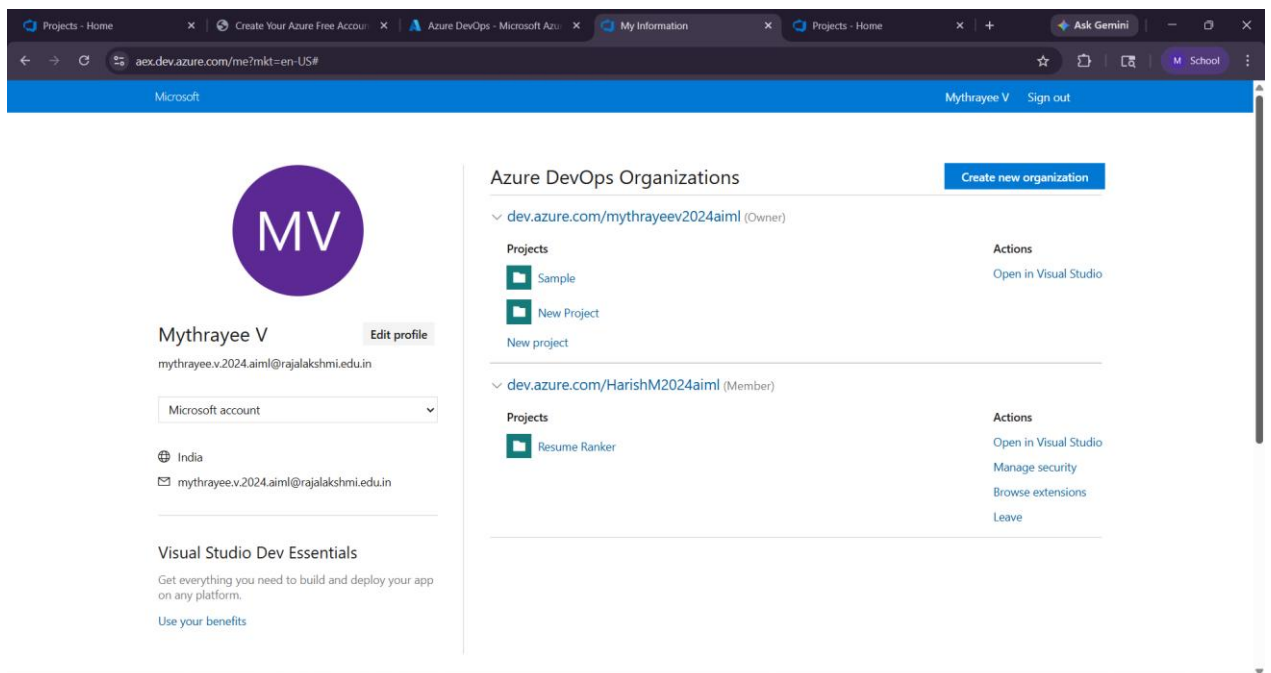
2. Azure home page:



3. Open DevOps environment in the Azure platform by typing **Azure DevOps Organizations** in the search bar.



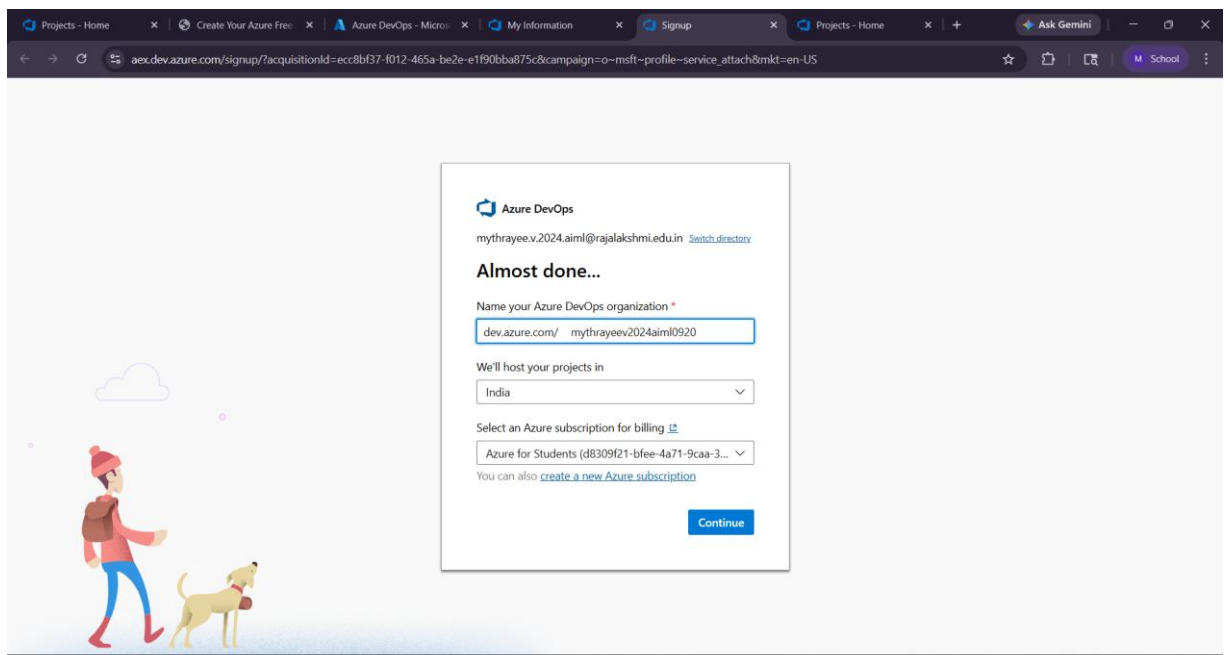
4. Click on the My Azure DevOps Organization link and create an organization and you should be taken to the Azure DevOps Organization Home page.

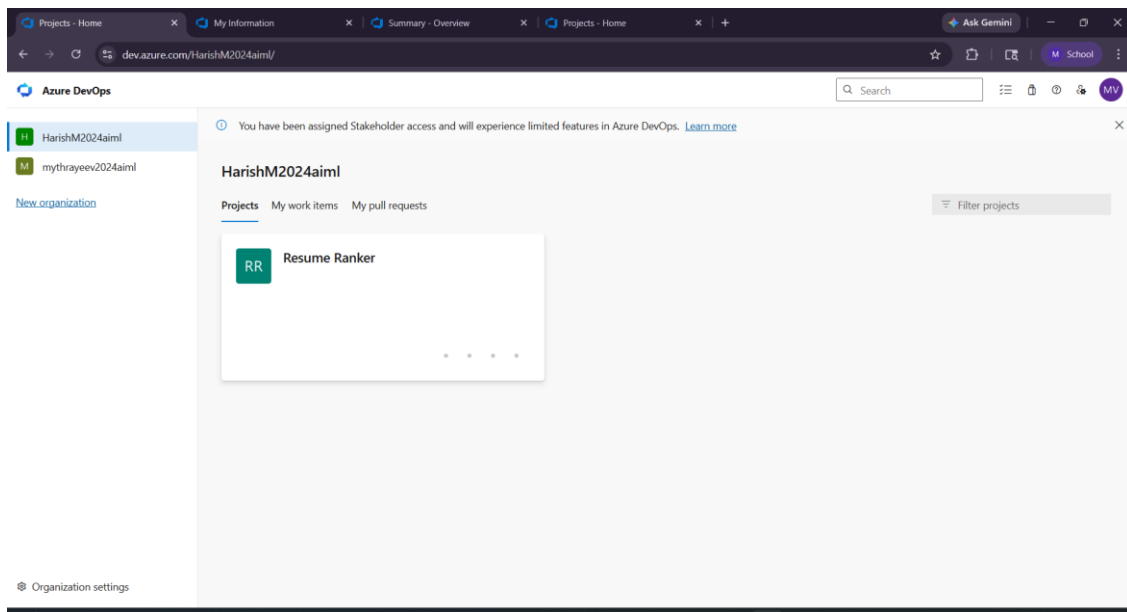


AZURE DEVOPS PROJECT SETUP AND USER STORY MANAGEMENT:

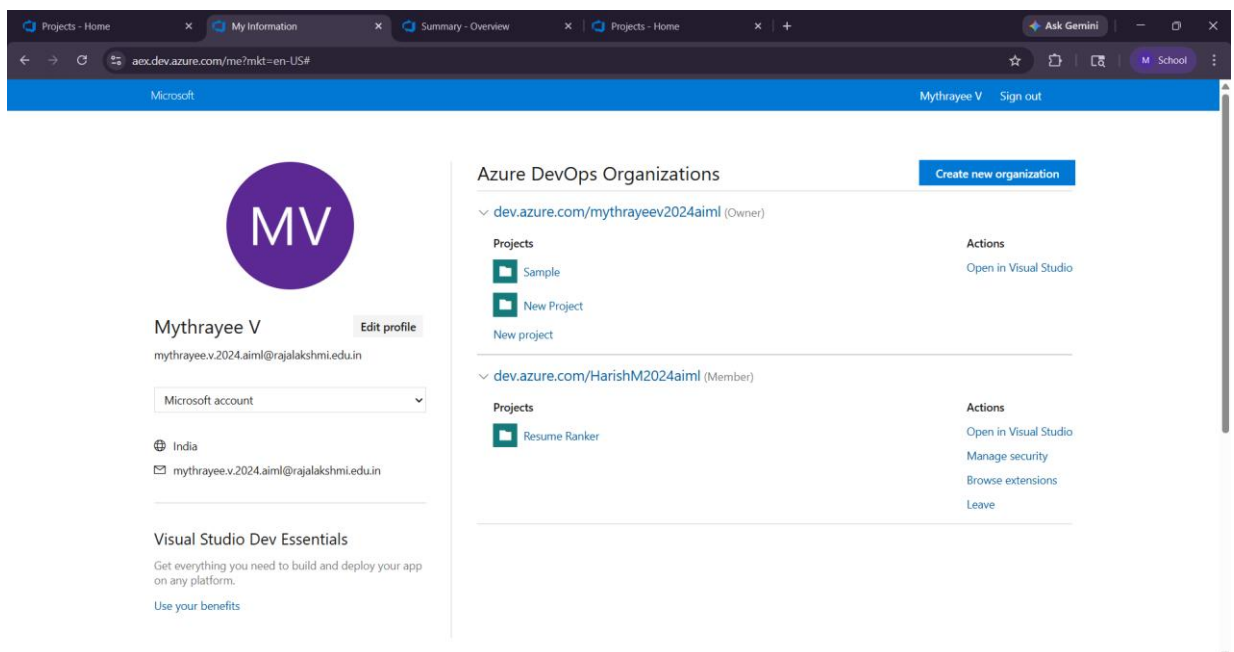
1. Create the First Project in Your Organization

- After the organization is set up, you'll need to create your first project. This is where you'll begin to manage code, pipelines, work items, and more.
- On the organization's Home page, click on the New Project button.
- Enter the project name, description, and visibility options: Name: Choose a name for the project (e.g., LMS).
 - Description: Optionally, add a description to provide more context about the project.
 - Visibility: Choose whether you want the project to be Private (accessible only to those invited) or Public (accessible to anyone).
- Once you've filled out the details, click Create to set up your first project.

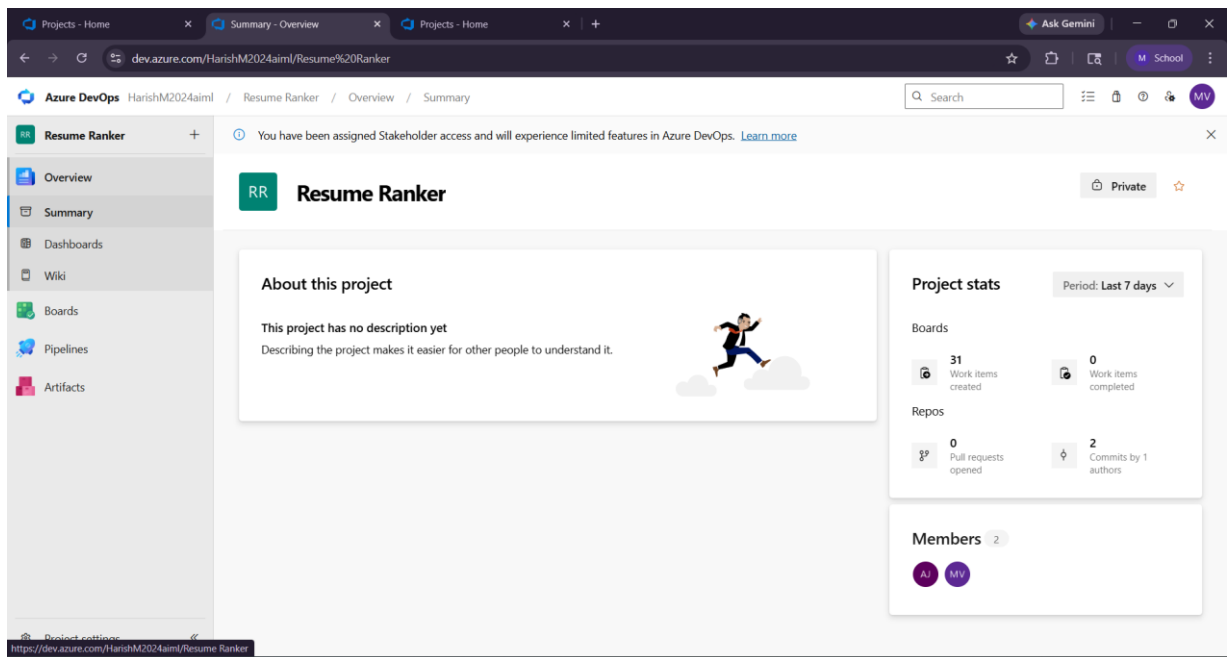




2. Once logged in, ensure you are in the correct organization. If you're part of multiple organizations, you can switch between them from the top left corner (next to your user profile). Click on the Organization name, and you should be taken to the Azure DevOps Organization Home page.

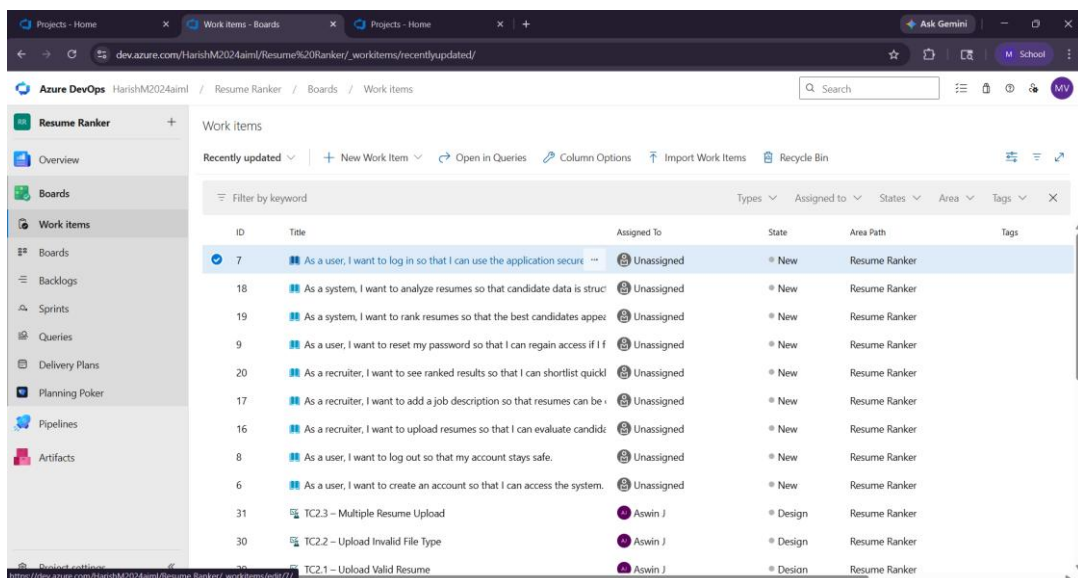


3. Project dashboard



4. To manage user stories:

- a. From the left-hand navigation menu, click on Boards. This will take you to the main Boards page, where you can manage work items, backlogs, and sprints.
- b. On the work items page, you'll see the option to Add a work item at the top. Alternatively, you can find a + button or Add New Work Item depending on the view you're in. From the Add a work item dropdown, select User Story. This will open a form to enter details for the new User Story.



Projects - Home

Resume Ranker Team Epics Ba

Projects - Home

Ask Gemini

dev.azure.com/HarishM2024aiml/Resume%20Ranker/_backlogs/backlog/Resume%20Ranker%20Team/Epics

Search

Menu icons

MV

Resume Ranker

Overview

Boards

Work items

Boards

Backlogs

Sprints

Queries

Delivery Plans

Planning Poker

Pipelines

Artifacts

Project settings

As a stakeholder, you can access the backlog, task and Kanban boards, work items and manage approvals for Releases. [Learn more](#)

Resume Ranker Team

New Work Item

View as Board

Column Options

Backlog

Analytics

	Order	Work Item Type	Title	State	Effort	Busin...	Va
1		Epic	User Authentication	New			Bus:
		Feature	User registration (sign up)	New			Bus:
		User Story	As a user, I want to create an account so that I can acc...	New			Bus:
		Feature	User login (email & password)	New			Bus:
		User Story	As a user, I want to log in so that I can use the applica...	New			Bus:
		Feature	Logout functionality	New			Bus:
		User Story	As a user, I want to log out so that my account stays s...	New			Bus:
2		Feature	Password reset	New			Bus:
		User Story	As a user, I want to reset my password so that I can re...	New			Bus:
		Epic	Resume Ranking	New			Bus:
		Feature	Upload resumes	New			Bus:
		User Story	As a recruiter, I want to upload resumes so that I can e...	New			Bus:

Epics

Planning

Drag and drop work items to include them in a sprint.

Resume Ranker Team backlog

Iteration 1 5/3/2026 - 5/16/2026
Planned Effort: 1810 working days
4

Iteration 2 5/17/2026 - 5/30/2026
Planned Effort: 810 working days
2

Iteration 3 6/1/2026 - 6/12/2026
Planned Effort: 1310 working days
1

Epic 1: Project Setup and Environment

Feature: Development Environment Setup

User Story 1:

As a developer, I want to set up the project environment so that I can begin development.

Tasks:

- Create project folder resume-ranker
- Initialize Python virtual environment
- Install required dependencies (Flask, pandas, numpy, nltk)
- Create requirements.txt file

User Story 2:

As a developer, I want to create a structured project layout so that development is organized.

Tasks:

- Create directories: backend/, frontend/, tests/
- Create initial files (app.py, parser.py, ranking.py)
- Initialize version control (Git)

Epic 2: User Authentication

Feature: Account Management

User Story 3:

As a user, I want to create an account so that I can access the system.

Tasks:

- Design registration form
- Implement signup API
- Store user data in database

User Story 4:

As a user, I want to log in and log out so that I can securely access the system.

Tasks:

- Create login interface
- Validate credentials
- Implement session/logout functionality

Epic 3: Resume Upload System

Feature: Resume Management

User Story 5:

As a recruiter, I want to upload resumes so that candidates can be evaluated.

Tasks:

- Create file upload UI
- Support PDF/DOCX formats
- Store resumes on server

User Story 6:

As a recruiter, I want to view uploaded resumes so that I can manage them.

Tasks:

- Display uploaded files
- Allow delete/download options
- Maintain resume list

Epic 4: Job Description Management

Feature: Job Input

User Story 7:

As a recruiter, I want to add a job description so that resumes can be matched.

Tasks:

- Create job description form
- Save data in database
- Validate input fields

User Story 8:

As a recruiter, I want to edit or delete job descriptions so that data stays updated.

Tasks:

- Implement edit functionality
- Enable delete option
- Update database records

Epic 5: Resume Parsing Engine

Feature: Data Extraction

User Story 9:

As a system, I want to extract text from resumes so that data can be processed.

Tasks:

- Integrate parsing libraries
- Extract raw text
- Handle different formats

User Story 10:

As a system, I want to clean and structure extracted data so that analysis is accurate.

Tasks:

- Remove unwanted characters
- Normalize text
- Store structured data

Epic 6: Skill Extraction

Feature: Candidate Analysis

User Story 11:

As a system, I want to identify skills in resumes so that candidates can be evaluated.

Tasks:

- Define skill keywords
- Apply NLP techniques
- Extract relevant skills

User Story 12:

As a system, I want to categorize skills so that matching becomes easier.

Tasks:

- Group skills (technical, soft skills)
- Store categorized data
- Optimize keyword mapping

Epic 7: Experience Analysis

Feature: Experience Detection

User Story 13:

As a system, I want to extract work experience so that candidates can be assessed.

Tasks:

- Identify experience sections
- Extract years and roles
- Store experience details

User Story 14:

As a system, I want to calculate total experience so that ranking is accurate.

Tasks:

- Sum experience years
- Handle overlapping roles
- Normalize values

Epic 8: Resume Matching

Feature: Matching Engine

User Story 15:

As a system, I want to match resumes with job descriptions so that relevant candidates are identified.

Tasks:

- Compare skills with job requirements
- Apply matching algorithm
- Generate similarity score

User Story 16:

As a system, I want to filter irrelevant resumes so that only suitable candidates remain.

Tasks:

- Set threshold score
- Remove low matches
- Optimize filtering logic

Epic 9: Candidate Ranking

Feature: Ranking System

User Story 17:

As a system, I want to assign scores to candidates so that they can be evaluated.

Tasks:

- Define scoring criteria
- Calculate candidate scores
- Store results

User Story 18:

As a system, I want to sort candidates so that the best candidates appear first.

Tasks:

- Implement sorting algorithm
- Handle equal scores
- Optimize ranking logic

Epic 10: Results Dashboard

Feature: Results Visualization

User Story 19:

As a recruiter, I want to view ranked candidates so that I can shortlist them.

Tasks:

- Design results page
- Display ranked list
- Show candidate scores

User Story 20:

As a recruiter, I want to filter and search results so that I can quickly find candidates.

Tasks:

- Add search functionality
- Implement filters
- Improve UI responsiveness

Project Planning

The project was developed using Agile methodology to ensure iterative development and efficient task management. The system was designed in modular components including resume upload, parsing engine, skill extraction, matching engine, backend API, and frontend dashboard.

Tools and Technologies

- Backend: Python, Flask
- Data Processing: Pandas, NumPy, NLP libraries (NLTK/spaCy)
- Frontend: HTML, CSS, JavaScript
- Visualization: Chart.js
- Database: SQLite / MongoDB
- Testing: pytest
- Project Management: Azure DevOps

Team Structure

The project was completed by a team of two members with shared and flexible responsibilities:

- **Full-Stack Developer 1:** Handled project planning, backend development (resume parsing, skill extraction, matching engine, and API), and contributed to frontend integration
- **Full-Stack Developer 2:** Focused on frontend design (UI and dashboard), API integration, testing, and ensuring system usability and performance

Sprint Planning

The project was divided into 3 sprints, each with specific objectives.

Sprint 1: Setup and Core Development

Objectives: Establish environment and implement core resume processing logic

Tasks:

- Setup project structure and development environment
- Implement resume upload functionality
- Develop resume parsing module (extract text from PDF/DOCX)
- Implement basic skill extraction logic

Sprint 2: Matching Engine and API Development

Objectives: Build matching logic and expose system functionality

Tasks:

- Develop job description input module
- Implement resume-job matching algorithm
- Build Flask API endpoints (upload, analyze, results)
- Integrate backend modules
- Write unit tests using pytest

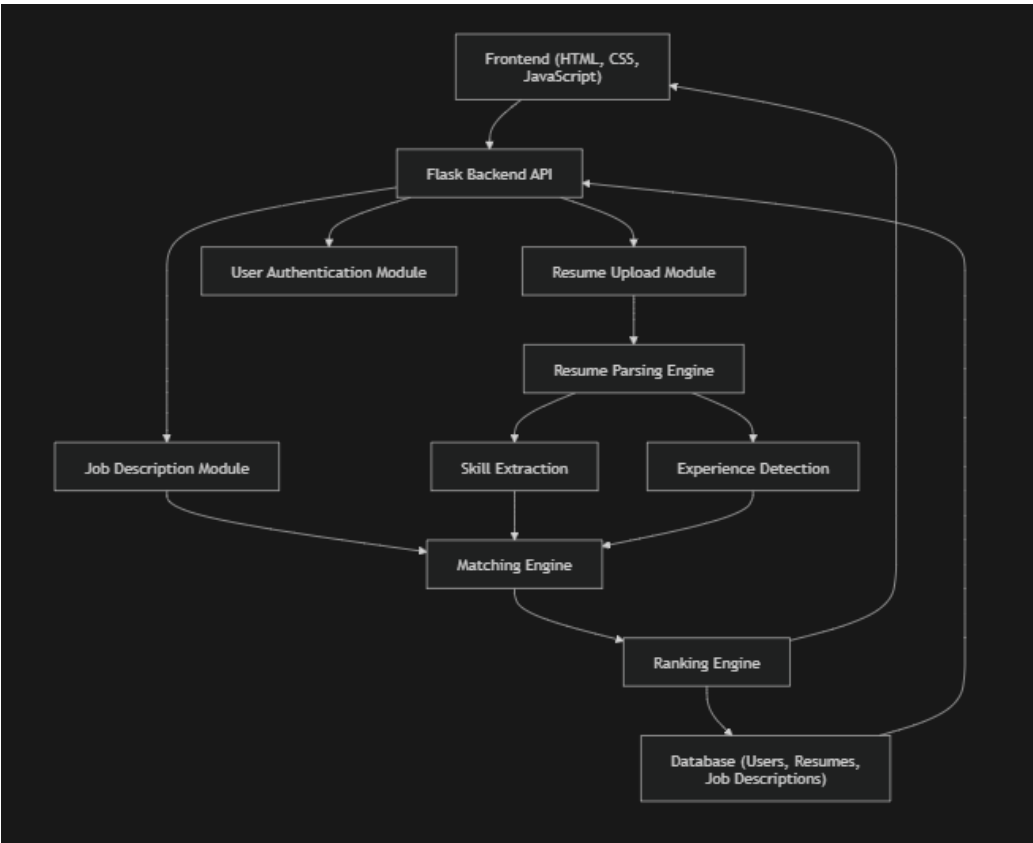
Sprint 3: Frontend and Finalization

Objectives: Develop user interface and complete system

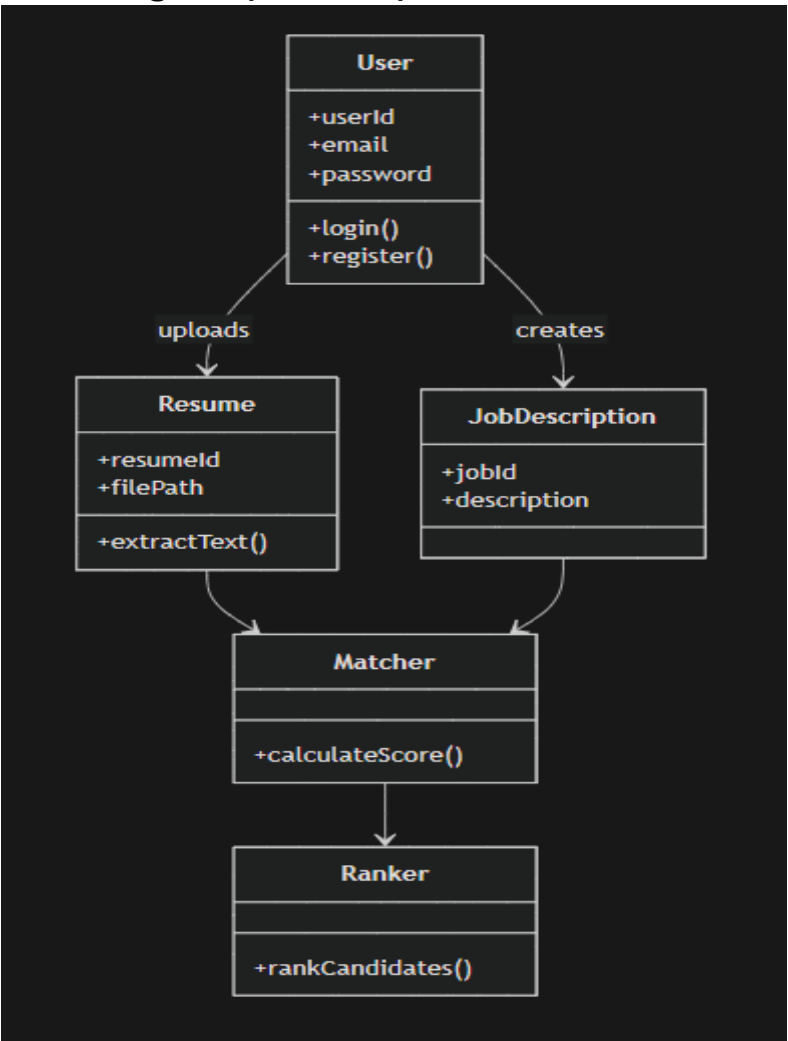
Tasks:

- Develop frontend dashboard (HTML, CSS, JavaScript)
- Display ranked candidates and scores
- Integrate Chart.js for visualization
- Implement filtering and search features
- Perform final testing and documentation

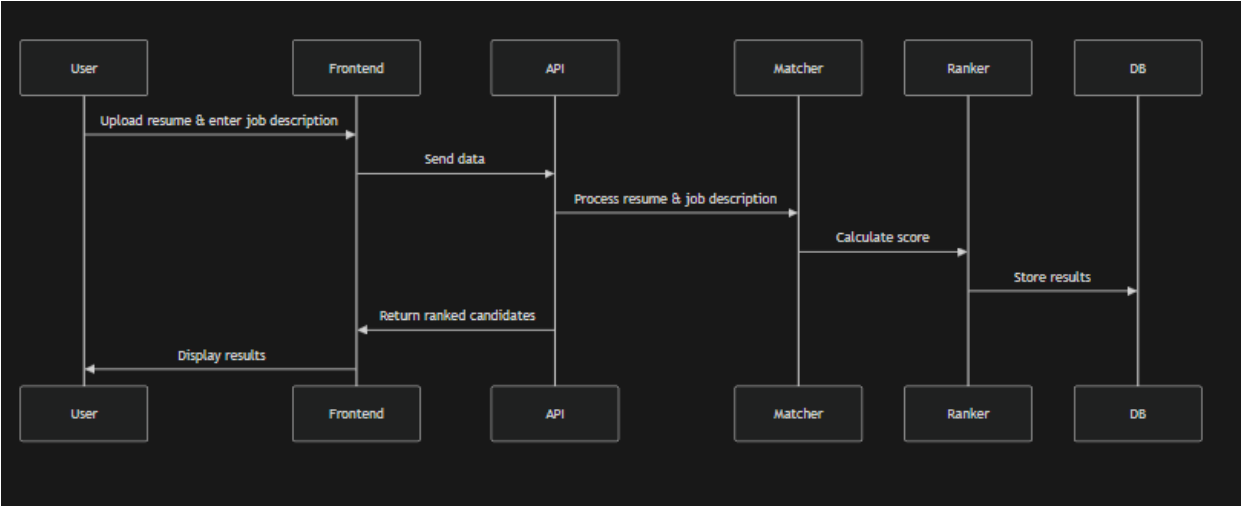
Architecture Diagram



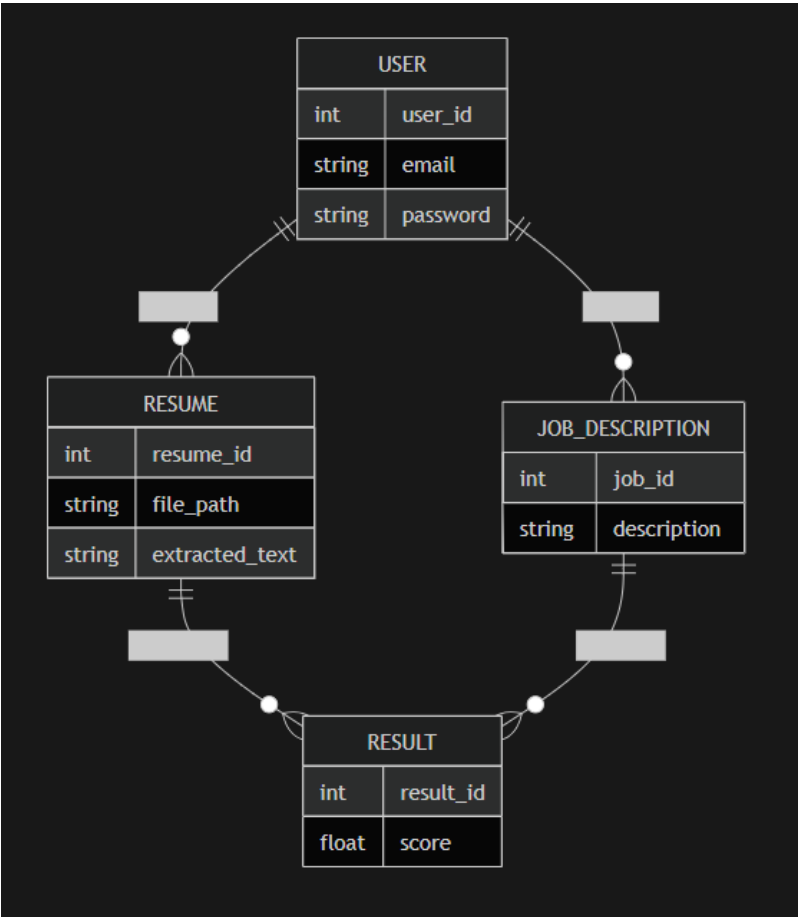
Class Diagram (Backend)



Sequence Diagram



ER Diagram



Project structure diagram:

